

Troubleshooting en DevOps

Guía de referencia: qué es, por qué importa, metodología, herramientas y buenas prácticas

1. Definición

El troubleshooting es el proceso sistemático de detectar, diagnosticar, aislar y resolver fallas o comportamientos anómalos en sistemas, aplicaciones, infraestructura o pipelines. En DevOps cobra especial importancia porque los sistemas son complejos, distribuidos y cambian constantemente: despliegues frecuentes, infraestructura como código, microservicios y contenedores.

No es simplemente “arreglar lo que está roto”: es un enfoque metódico para entender por qué algo falló, evitando parches superficiales que dejan el problema latente.

2. Por qué es central en DevOps

DevOps busca entregar software rápido y de forma confiable mediante integración y despliegue continuos (CI/CD), automatización e infraestructura como código. Esta velocidad y complejidad generan más puntos de falla posibles:

- Cambios de código constantes
- Múltiples entornos (dev, staging, producción)
- Dependencias externas (APIs, bases de datos, servicios cloud)
- Infraestructura dinámica (contenedores, autoescalado, orquestadores)

Por eso, saber diagnosticar rápido reduce el **MTTR** (Mean Time To Recovery), una métrica clave en DevOps y SRE (Site Reliability Engineering).

3. Categorías comunes de problemas

Pipelines CI/CD — Builds que fallan por dependencias rotas, tests intermitentes (“flaky tests”), deploys que se cuelgan o hacen rollback automático, permisos insuficientes en runners/agents.

Infraestructura — Servidores o pods caídos, problemas de red (DNS, firewalls, latencia), contenedores que no arrancan o reinician en loop (CrashLoopBackOff), falta de recursos (CPU, memoria, disco).

Aplicación — Errores 500 en producción, memory leaks, performance degradado bajo carga, excepciones no manejadas.

Configuración — Variables de entorno mal seteadas, secretos o credenciales faltantes/expirados, archivos de configuración desincronizados entre entornos.

Base de datos — Queries lentas o bloqueos (locks), conexiones agotadas (connection pool exhausted), migraciones fallidas.

4. Metodología estructurada

Un enfoque sistemático reduce el tiempo de resolución y evita soluciones parche. El ciclo típico tiene siete etapas y termina realimentando la primera:

- 1

Detectar
Vía alertas, monitoreo o reporte de usuario
- 2

Reproducir
El problema si es posible, en un entorno controlado
- 3

Recolectar evidencia
Logs, métricas, trazas (traces), eventos recientes
- 4

Formular hipótesis
¿Qué cambió recientemente? (deploy, config, dependencia)
- 5

Aislar la causa
Dividir el sistema en partes y descartar una por una
- 6

Verificar y aplicar
Probar en entorno seguro, luego aplicar fix, rollback o mitigación
- 7

Postmortem / RCA
Documentar qué pasó, por qué, y cómo evitarlo (cultura blameless)

El postmortem realimenta el ciclo: lo aprendido mejora la detección y la respuesta ante el próximo incidente.

5. Herramientas típicas por categoría

Categoría	Herramientas comunes
Logs	ELK Stack (Elasticsearch, Logstash, Kibana), Loki, Splunk
Métricas	Prometheus, Grafana, Datadog
Trazas distribuidas	Jaeger, OpenTelemetry, Zipkin
CI/CD	GitHub Actions, GitLab CI, Jenkins, CircleCI
Contenedores / Orquestación	Docker, Kubernetes (kubectl logs, kubectl describe)

Categoría	Herramientas comunes
Infraestructura como código	Terraform, Ansible
Cloud	AWS CloudWatch, GCP Logging, Azure Monitor
Red	ping, traceroute, curl, netstat, dig

6. Comandos y técnicas prácticas frecuentes

Ver logs de un contenedor

```
docker logs <container_id>
```

Ver estado y eventos de un pod en Kubernetes

```
kubectl describe pod <pod_name>  
kubectl logs <pod_name> --previous
```

Ver uso de recursos

```
top / htop  
docker stats
```

Verificar conectividad de red

```
curl -v https://servicio.com  
telnet host puerto
```

Ver procesos y puertos abiertos

```
netstat -tulpn  
lsof -i :puerto
```

7. Principios clave para troubleshooting efectivo

- **No asumir, verificar:** cada hipótesis debe confirmarse con datos, no con intuición.
- **Un cambio a la vez:** si modificás varias cosas simultáneamente, no sabrás cuál resolvió el problema.
- **Revisar lo que cambió recientemente:** la mayoría de los incidentes en producción vienen de un deploy o cambio de configuración reciente.
- **Tener observabilidad:** sin logs/métricas suficientes es casi imposible diagnosticar bien.
- **Automatizar diagnósticos repetitivos:** scripts o dashboards que muestren rápido el estado del sistema.

8. Relación con SRE

En equipos de Site Reliability Engineering, el troubleshooting está formalizado con conceptos como:

- **SLI/SLO/SLA:** indicadores y objetivos de nivel de servicio, para saber cuándo algo “realmente” está mal
- **Runbooks:** guías paso a paso para resolver incidentes conocidos
- **On-call:** rotación de guardia para responder incidentes en tiempo real

Guía elaborada como referencia rápida de troubleshooting en DevOps.